

Sovereign Mathematical Optimization Engine

VX03 System Architecture, Mathematical Foundations & Implementation Specification

Core Architectural Tenet: "ML decides HOW the problem should be attacked. The mathematical solver determines and verifies WHAT the solution is. ML must never become the authority for mathematical correctness."

1. System Executive Summary & Purpose

The **Sovereign Mathematical Optimization Engine (VX03)** is a sparse-first, heterogeneous CPU-GPU optimization system engineered entirely from mathematical first principles. It delivers production-grade linear (LP), mixed-integer linear (MILP), convex quadratic (QP), and mixed-integer quadratic (MIQP) optimization capabilities without wrapping or depending on external blackbox solver engines (such as Gurobi, CPLEX, SCIP, or HiGHS).

Every mathematical component—including standard-form slack transformations, Ruiz equilibration, LU basis updates, Mehrotra predictor-corrector barrier updates, Gomory mixed-integer cut generators, and dual KKT solution certifiers—is natively implemented and verified within the repository.

2. End-to-End System Pipeline

The solver operates as a 10-stage sequential pipeline that transforms raw model inputs into independently certified optimal solution vectors:

Pipeline Stage	Description & Operations
1. Input Parsing	Parses raw MPS (Fixed/Free) or CPLEX LP files into a unified OptimizationModel object.
2. Validation	Performs sanity checks, detects bound conflicts, and classifies problem type (LP, MILP, QP, MIQP).
3. Sparse Matrix Engine	Constructs dual Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) representations.
4. Presolve & Scaling	Applies singleton/empty reductions, forcing rules, and iterative Ruiz matrix equilibration.
5. Feature Extraction	Extracts a 28-dimensional structural problem descriptor vector for machine learning dispatch.
6. ML Strategy Engine	Selects optimal solver family (Simplex vs IPM) and hardware (CPU vs GPU) with fallback safety.
7. Sovereign Core	Solves problem using native Revised Simplex, Mehrotra IPM, Branch-and-Cut, or Active-Set QP.
8. Heterogeneous Runtime	Dispatches matrix math to CPU multithreading or PyTorch GPU tensors with automatic fallback.
9. Postsolve Unwinding	Reverts presolve reduction stack to map solution back into canonical user variable space.
10. Independent Trust Audit	Independently audits primal feasibility, integrality margin, objective, and KKT duals.

3. Detailed Subsystem Implementation Specifications

3.1 Universal Model & Sparse Matrix Engine

Located in `sovereign_opt/model/` and `sovereign_opt/sparse/`. Provides foundational data structures:

- **OptimizationModel** (`model.py`): Encapsulates variables, constraints, and objective senses. Handles conversion to standard matrix forms $Ax \leq b$ or $Ax = b$.
- **Dual CSR/CSC Matrix** (`matrix.py`): Compressed Sparse Row (CSR) optimized for constraint evaluation (Ax), and Compressed Sparse Column (CSC) optimized for column pricing and transpose multiplication ($A^T y$).
- **Ruiz Equilibration** (`scaling.py`): Iteratively computes diagonal scaling matrices D_r and D_c such that row and column infinity-norms converge to 1.0: $\|D_r A D_c\|_\infty \approx 1.0$, drastically reducing numerical ill-conditioning.

3.2 Presolve Reduction & Postsolve Reconstruction

Located in `sovereign_opt/presolve/`. Eliminates redundant variables and constraints before solver execution:

- **Presolver** (`presolver.py`): Executes fixed variable elimination, empty row/col removal, singleton bound tightening, doubleton aggregation, and forcing constraint reductions.
- **Postsolve Mapper** (`postsolve.py`): Inverts the reduction stack post-solve to reconstruct canonical variable values $x^* \in \mathbb{R}^n$ and dual multipliers.

3.3 28-Dimensional AI Strategy Engine

Located in `sovereign_opt/ml/`. Computes structural problem features to guide algorithm selection:

- **Feature Extractor** (`features.py`): Extracts 28 structural metrics including matrix shape, aspect ratio (m/n), sparsity density, coefficient dynamic range, variable bound ratios, and integer fraction.
- **Strategy Harness** (`strategy.py`): Recommends optimal solver (Revised Simplex vs IPM vs Active-Set QP) and hardware backend. Features a deterministic safety harness that falls back to standard defaults if confidence is below threshold.

3.4 Sovereign Solver Core

Located in `sovereign_opt/solvers/`. Houses native solver algorithms:

- **Linear Programming (LP)**: `simplex_engine.py` implements Two-Phase Revised Simplex with LU basis updates, Devex and Dual Steepest-Edge pricing, and Harris ratio testing. `interior_point.py` implements Mehrotra Predictor-Corrector Primal-Dual IPM with Homogeneous Self-Dual (HSD) embedding and crossover.
- **Mixed-Integer LP (MILP)**: `branch_bound.py` manages a min-heap Best-Bound search tree. `branching.py` provides Most-Fractional and ML-guided pseudocost branching. `cuts.py` generates Gomory Mixed-Integer (GMI) and Knapsack cover cuts. `heuristics.py` runs Feasibility Pump, RINS, and Diving heuristics.
- **Quadratic Programming (QP/MIQP)**: `active_set.py` and `interior_point_qp.py` solve convex QPs via KKT augmented system factorization with active-set polishing. MIQP is solved via Branch-and-Bound over QP relaxations.

3.5 Independent Mathematical Trust Validator

Located in `sovereign_opt/validation/validator.py`. Recomputes all verification metrics directly from raw model equations without trusting internal solver flags. Validates primal residuals $\|(Ax - b)\|_\infty$, bound violations, integrality margins $|x - \text{round}(x)|$, recomputes objectives, and verifies KKT dual conditions $z = \nabla f(x) - A^T y$ to issue an immutable `ValidationCertificate`.

4. Key v1.0 vs v2.0 Engine Upgrades & Measured Results

Optimization Area	v1.0 Measured Behaviour	v2.0 Certified Behaviour
Free Variables	Simplex -10000, IPM 0 (both labeled optimal)	-50000 exact solution, KKT certified
Maximization Presolve	Empty columns fixed at wrong bound	Sense-normalized reductions (150/150 correct)
Netlib AFIRO LP	Unbounded transcription; IPM diverged to -6e53	-464.7531428571 certified optimal (error $\leq 1e-13$)
MIQP Problems	Quadratic terms dropped; sent to simplex	Branch-and-Bound over convex QP relaxations
Refinery QP	Active set capped out (-5,889,292)	-5,903,161.066 certified optimal (IPM: 11 iters)
Unit Commitment MILP	Reported incorrect gap; crashed >4 periods	4h / 8h / 24h proven optimal, gap 0 (0.18s)
Trust Verification	Feasibility checks only	Dual KKT optimality + strong duality gap check

5. Full-Stack Workstation Interfaces & Web Visualizer

The system includes a dual interface suite: a **FastAPI REST Server** (server.py) paired with a modern **Next.js Terminal Visualizer** (frontend/app/page.tsx), and a **Rich Terminal CLI** (sovereign_opt/cli/app.py).

The Next.js dashboard features an interactive 6-stage telemetry workflow visualizer:

- Stage 01 (Topology & Ruiz):** Interactive 2D matrix spy plot of non-zero entries A_{nn} and Ruiz norm convergence.
- Stage 02 (Presolve Diff):** Side-by-side dimension reduction metrics (vars, constraints, non-zeros).
- Stage 03 (AI Meta-Strategy):** Visual solver probability distribution and 28-dim feature attributions.
- Stage 04 (Solver Dynamics):** Real-time SVG polyline objective convergence trajectory and interactive Branch-and-Bound tree explorer.
- Stage 05 (Postsolve Map):** Variable unwind mapper tracking origin (OPTIMIZED_IN_CORE vs FIXED_IN_PRESOLVE).
- Stage 06 (Trust Audit):** Mathematical validation certificate badge with primal, dual, and integrality margin metrics.

6. Quickstart & Execution Commands

```
# 1. Launch Master Full-Stack Demo (FastAPI REST Server + Next.js Web UI)
python run_demo.py

# 2. Run Industrial Case Studies via Rich Terminal CLI
python -m sovereign_opt.cli.app demo --preset refinery_blending_lp
python -m sovereign_opt.cli.app demo --preset power_unit_commitment

# 3. Solve Custom MPS / LP File
python -m sovereign_opt.cli.app solve model.mps --algorithm auto

# 4. Run Automated Test Suite & Benchmarks
python -m pytest -v tests/
python -m benchmarks.run_benchmarks
```